

Backend Engineering & REST APIs

Core Revision Notes

Q1. What is an API? Explain the concept of an Interface as a contract.

An API (Application Programming Interface) is an abstraction defining interaction mechanics between software systems:

- **Input:** Expected parameters, HTTP headers, query strings, or request body.
- **Processing:** Internal business logic, database queries, or external computation.
- **Output:** Guaranteed response data format, status codes, and payload schema.
- **Encapsulation:** Consumers interact exclusively via the contract, allowing developers to refactor internal implementations without breaking client applications.

Q2. What are the types of APIs?

- **In-Process / Language APIs:** Embedded within the runtime environment; invoked locally in memory with zero network overhead (e.g., `setTimeout()`, `document.getElementById()`).
- **Network / Web APIs:** Hosted on remote machines accessed over networks via HTTP (e.g., CoWIN API for vaccine queries, Fake Store API for mock product catalogs).

Q3. What is the difference between a Library and a Framework?

The Cooking Analogy: When preparing food, you buy raw ingredients and packaged spice mixes instead of farming wheat from scratch. Similarly, common software problems (auth, routing, ORM) use pre-built building blocks.

Feature	Library	Framework
Scope	Single, focused capability	Complete architectural scaffolding
Control Flow	You call the code directly	Inversion of Control (IoC) ; it calls you
Conventions	Unopinionated; custom layout	Highly opinionated; enforces structure
Examples	React.js, Express.js, Java Collections	Ruby on Rails, Django, Spring Boot

Q4. What is REST? Guidelines vs. Protocols.

REST (Representational State Transfer) is an architectural style for distributed web systems.

- **Protocol:** Mandatory network laws (TCP, HTTP). Violations break data transmission.
- **Guidelines:** Recommended best practices. Violations don't crash networks, but compliance ensures consistency, maintainability, and predictable contracts.
- **REST API:** An API whose design adheres to REST architectural constraints.

Q5. Compare major API paradigms across the industry.

Paradigm	Format	Focus	Industry Use Case
REST	JSON	Resources (Nouns)	90–95% of web services (Google, Microsoft, Flipkart).
gRPC	Protobuf	Actions (RPC)	High-throughput, low-latency microservices (Google internal).
GraphQL	JSON	Client queries	Flexible client-driven apps; prevents over/under-fetching.
SOAP	XML	Strict XML	Legacy enterprise banking and legacy government services.

Q6. What are the core REST URL design conventions?

- **Resources vs. Actions:** Endpoints represent **nouns**, not verbs. The verb is supplied exclusively by the HTTP method.
 - ❌ *Avoid (RPC):* `POST /createBlog`, `GET /getBlogById?id=10`
 - ✅ *Follow (REST):* `POST /blogs`, `GET /blogs/10`
- **Pluralized Naming:** Always use plural nouns for collections (`/blogs` , `/users`).

Q7. Map standard CRUD operations to HTTP methods.

Method	CRUD Action	Endpoint Example	Description
GET	Read (All)	<code>GET /blogs</code>	Retrieve a collection of all blogs.
GET	Read (Single)	<code>GET /blogs/:id</code>	Retrieve a specific blog by dynamic path ID.
POST	Create	<code>POST /blogs</code>	Create a new blog via request payload.
PUT	Full Update	<code>PUT /blogs/:id</code>	Completely overwrite/replace the resource.
PATCH	Partial Update	<code>PATCH /blogs/:id</code>	Update designated fields of the resource.
DELETE	Delete	<code>DELETE /blogs/:id</code>	Remove the specified resource.

Q8. How are relationships modeled via nesting? What is the rule of thumb?

Parent-child relationships are represented using hierarchical path segments:

- **Get all comments of blog 13:** `GET /blogs/13/comments`
- **Get comment 3 on blog 22:** `GET /blogs/22/comments/3`
- **Create comment on blog 10:** `POST /blogs/10/comments`
- **Nesting Limit:** Avoid nesting deeper than **2 to 3 levels** (e.g., avoid `/a/:id/b/:id/c/:id/d/:id`) to prevent brittle, overcomplicated routes.

Q9. What are the three ways to send data to an API?

- **1. Request Body:** Encapsulates structured data (primarily JSON) inside the HTTP request payload. Used in `POST`, `PUT`, and `PATCH` requests.
Security Note: Sensitive credentials (passwords, payment info) must always travel in the request body to avoid exposure in URL server logs, proxies, and browser caches.
- **2. URL Parameters (Path Params):** Variable segments in the path used to locate a specific resource (e.g., `12` in `/blogs/12`).
- **3. Query Parameters:** Key-value pairs appended after `?` and joined by `&` used for filtering, sorting, or pagination (e.g., `/products?category=electronics&limit=10`).

Q10. Explain common HTTP Status Codes in REST APIs.

- **200 OK:** Request succeeded; response payload returned.
- **201 Created:** New resource successfully created (standard for `POST`).
- **204 No Content:** Succeeded, but no response body returned (common for `DELETE`).
- **400 Bad Request:** Malformed request payload, syntax error, or missing required fields.
- **401 Unauthorized:** Missing or invalid authentication credentials.
- **404 Not Found:** The requested resource URI does not exist on the server.
- **500 Internal Server Error:** Unhandled server-side crash or uncaught runtime exception.

Q11. What is API Versioning and why is it important?

Explicitly specifying versions in endpoint paths (e.g., `/v1/submissions`, `/v2/submissions`):

- **Prevents Breaking Changes:** Shields existing client integrations from sudden schema changes.
- **Backward Compatibility:** Older mobile or web clients function uninterrupted without forced updates.
- **Safe Rollouts & Zero Downtime:** Deploy `/v2` while `/v1` remains online; clients can fall back instantly if defects arise.
- **Deprecation Windows:** Enables parallel maintenance periods (e.g., 6–12 months) before decommissioning.

Q12. What is a CRUD API? How do modern frameworks support it?

CRUD stands for **Create**, **Read**, **Update**, and **Delete**. A CRUD API provides the complete lifecycle of persistence endpoints mapping to `POST`, `GET`, `PUT/PATCH`, and `DELETE`.

Frameworks like **Ruby on Rails** and **Django REST Framework** leverage strict REST conventions to automatically generate routes, controllers, and serializers with minimal boilerplate code.